

Programação Funcional

Classes em Haskell

Eliseu C. Miguel

Universidade Federal de Alfenas

11 de agosto de 2025

Tópicos

- 1 Introdução às Classes de Tipos
- 2 Classe Eq
- 3 Classe Ord
- 4 Classe Show
- 5 Derivação Automática
- 6 Referências Bibliográficas

Tópicos

- 1 Introdução às Classes de Tipos
- 2 Classe Eq
- 3 Classe Ord
- 4 Classe Show
- 5 Derivação Automática
- 6 Referências Bibliográficas

Tópicos

- 1 Introdução às Classes de Tipos
- 2 Classe Eq
- 3 Classe Ord
- 4 Classe Show
- 5 Derivação Automática
- 6 Referências Bibliográficas

Tópicos

- 1 Introdução às Classes de Tipos
- 2 Classe Eq
- 3 Classe Ord
- 4 Classe Show
- 5 Derivação Automática
- 6 Referências Bibliográficas

Tópicos

- 1 Introdução às Classes de Tipos
- 2 Classe Eq
- 3 Classe Ord
- 4 Classe Show
- 5 Derivação Automática
- 6 Referências Bibliográficas

Tópicos

- 1 Introdução às Classes de Tipos
- 2 Classe Eq
- 3 Classe Ord
- 4 Classe Show
- 5 Derivação Automática
- 6 Referências Bibliográficas

O que são Classes de Tipos?

Classes de tipos em Haskell

Classes de tipos são uma forma de definir interfaces para tipos:

- Especificam operações que os tipos devem implementar
- Permitem polimorfismo ad-hoc (sobrecarga)
- Garantem consistência nas operações

Hoje focaremos em três classes fundamentais:

- **Eq** - para tipos comparáveis por igualdade
- **Ord** - para tipos ordenáveis
- **Show** - para tipos que podem ser representados como strings

Classe Eq: Definição

Definição da classe Eq

```
class Eq a where  
    (==) :: a -> a -> Bool  
    (/=) :: a -> a -> Bool
```

Características:

- Tipos que implementam Eq podem ser comparados por igualdade
- A implementação padrão define (/=) como negação de (==)
- Muitos tipos já são instâncias de Eq no Prelude

Implementando Eq para Tipos Customizados

Exemplo: Tipo Ponto

```
data Ponto = Ponto Double Double

instance Eq Ponto where
    (Ponto x1 y1) == (Ponto x2 y2) = x1 == x2 && y1
    == y2
```

Pontos importantes:

- Podemos derivar automaticamente: `data Ponto = ... deriving Eq`
- A implementação deve ser consistente (reflexiva, simétrica, transitiva)

Classe Ord: Definição

Definição da classe Ord

```
class Eq a => Ord a where  
  compare :: a -> a -> Ordering  
  (<), (<=), (>), (>=) :: a -> a -> Bool  
  max, min :: a -> a -> a
```

Observações:

- Ord herda de Eq (todo tipo Ord deve ser Eq)
- Tipos Ord podem ser ordenados e comparados
- Muitas funções de lista (sort, maximum, etc.) exigem Ord

Implementando Ord para Tipos Customizados

Exemplo: Tipo Produto

```
data Produto = Produto String Double

instance Ord Produto where
    compare (Produto _ p1) (Produto _ p2) = compare
p1 p2
```

Pontos importantes:

- Implementando apenas `compare`, as outras funções são derivadas automaticamente
- A ordenação deve ser consistente com a igualdade
- Podemos derivar automaticamente: `deriving (Eq, Ord)`

Classe Show: Definição

Definição da classe Show

```
class Show a where
  show :: a -> String
  showsPrec :: Int -> a -> ShowS
  showList :: [a] -> ShowS
```

Características:

- Converte valores em strings para exibição
- Fundamental para depuração e interação com usuário
- Usado pelo GHCi para mostrar resultados

Implementando Show para Tipos Customizados

Exemplo: Tipo Data

```
data Data = Data Int Int Int

instance Show Data where
    show (Data d m a) = show d ++ "/" ++ show m ++
"/" ++ show a
```

Observações:

- Podemos derivar automaticamente: `deriving Show`
- A representação deve ser clara e não ambígua
- `showsPrec` permite controle sobre precedência

Derivando Classes Automaticamente

Derivação automática

```
data Pessoa = Pessoa String Int  
    deriving (Eq, Ord, Show)
```

Vantagens:

- Implementações padrão consistentes
- Economiza tempo e código
- Boa para tipos simples

Limitações:

- Pode não ser adequado para tipos complexos
- Ordem de derivação importa (Ord requer Eq)

Referências Bibliográficas

Book [3] Slides [1] Book [2]



Vladimir Oliveira Di Iorio.

Programação Funcional - Tipos Básicos e Programas Simples em Haskell.

DPI - UFV.



Miran Lipovaca.

Learn You a Haskell for Great Good!

Lipovaca.



Simon Thompson.

Haskell: the Craft of Functional Programming.

Addison-Wesley, Harlow, England, 1997.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.
Ao contrário, é apenas um guia para lhe informar quais são os
tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos
sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a
bibliografia citada no plano de ensino da disciplina para estudar os
tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.